■ Hotel Booking Management System

Project Explanation & Architecture Guide for Trainers & Evaluators

1. Business Problem Statement (In Simple Words)

The Problem: Managing hotel reservations manually using paper registers or simple spreadsheets leads to frequent double-booking errors, incorrect billing, lack of visibility into room availability, and lost revenue tracking. Hotel managers need an easy, reliable system to handle reservations, track income, and manage customer records.

The Solution: This project delivers a centralized **Hotel Booking Management Web Application**. It allows administrators to log in securely, create new bookings with auto-calculated rates, view and search existing reservations, filter customer lists, track real-time revenue analytics through interactive charts, and export data for reporting.

2. How the System Works (End-to-End Workflow)

- **Step 1 (Authentication):** The administrator logs in using secure credentials (`admin@gmail.com` / `admin123`).
- **Step 2 (API Communication / Local Fallback):** The frontend sends HTTP requests (`POST`, `GET`, `DELETE`) via `requests` to a backend FastAPI server (`http://127.0.0.1:8000`). If offline, it uses local session data.
- **Step 3 (Booking Calculation Engine):** When adding a reservation, room rates and night stay durations are computed dynamically in real time.
- **Step 4 (Data Storage & Display):** Customer records are received as JSON, converted into `pandas` DataFrames, and rendered cleanly using Streamlit data tables.
- **Step 5 (Business Intelligence):** Data is aggregated to produce live room distribution pie charts and revenue bar charts using `plotly`.

3. Explanation of Each Project File

File / Component Name	Simple Role & Purpose in Project
finalproject.py / app.py	The complete frontend web application built using Streamlit. Handles login authentication, sidebar navigation, form entries, dashboard KPIs, and visual chart rendering.
FastAPI Backend (Local / Server)	The REST API backend responsible for processing database queries, saving booking payloads, managing OAuth tokens, and deleting records.
Database / Storage	Stores guest profile entries, room selections, check-in/out dates, and total payment amounts.
hotel_bookings.csv	Exported spreadsheet format generated directly from the frontend when downloading reservation records.
requirements.txt	Lists all external libraries required to run the project (`streamlit`, `requests`, `pandas`, `plotly`, `streamlit-option-menu`).

4. How the Frontend Works (Module Breakdown)

The user interface is engineered with **Streamlit** and custom CSS for a modern, responsive web layout. Here is how each module in the navigation menu functions:

• **Login Page:** Captures email/password credentials. Supports token-based authentication via FastAPI with local fallback mode for offline testing. Features session-based logout alerts.

• **Dashboard Page:** Displays high-level executive metric cards (Total Bookings, Revenue in ■, Unique Customers, Available Rooms), recent guest tables, and pie/bar charts.

• **Add Booking Form:** Interactive input form capturing guest details, room tier (Standard, Deluxe, Executive, Suite), and check-in/out dates. Auto-calculates total nights and price estimation.

• **View Bookings Page:** Displays the full guest database with live search filtering by name or room type, CSV download export, and a record deletion action.

• **Analytics Page:** Provides business performance metrics, room share breakdowns, guest count distribution, and revenue summaries.

• **Profile & Settings Pages:** Shows administrator details, statistics, password updates, theme options, and backup configuration.